Introduction

Every automation tool eventually asks you to answer the same question: is this value fixed, or does it change every time the workflow runs? Get that answer wrong and you don't get a crash — you get something worse. A workflow that runs perfectly, produces no errors, and quietly does the wrong thing.

This is the exact failure mode behind one of the most common n8n bugs: an email field with a hardcoded address that worked for the first signup and then silently misfired for every signup after it. Nothing broke. Nothing alerted anyone. The workflow just kept sending the wrong data to the wrong place, run after run.

Understanding n8n's expression syntax — the `{{ }}` blocks, the `$json` variable, and its long-range cousin `$node` — is what separates someone who can drag nodes onto a canvas from someone who can actually trust their automations in production. It's also one of those topics that looks trivial on the surface ("it's just curly braces") but hides a real mental model underneath, the same kind of scoping logic you'd find in variable resolution in any programming language. If you've ever debugged a closure capturing the wrong variable, this will feel familiar.

By the end of this article you'll know exactly how to turn a static field into a dynamic one, why `$json` means something different depending on which node you're standing in, when you need `$node` instead, and how to catch the single most common typo before it costs you twenty minutes of your life.

Problem Overview

Here's the scenario in plain terms: you're building a workflow that reacts to a new customer signup and sends them a welcome email. The first time you build this, it's tempting to just type an email address directly into the "To" field. It works, because you tested it with one specific person's data.

The problem is that a typed-in value is permanent. It doesn't know that the next person who signs up has a different email address. The field isn't asking "what's the email of the person who just signed up" — it's saying "always use this exact string, forever, no matter what."

What you actually want is a way to tell n8n: "grab the email address from the data that's flowing through this workflow right now." That's what an expression does. And once you understand how n8n decides what "right now" means at any given point in a workflow, this class of bug disappears entirely.

Example

Imagine three connected nodes:

- **Node A** — a webhook that receives signup data: `{ "first_name": "Maya", "last_name": "Chen", "email": "maya@example.com" }`
- **Node B** — a node that adds a `signup_date` field to that same item
- **Node C** — the "Send Email" node, where you configure the "To" field

If you're configuring the "To" field in Node C and you write:

```
{{ $json.email }}
```

n8n resolves this to `maya@example.com` — but only because Node B passed that field along unchanged from Node A. If Node B had transformed or dropped the `email` field, `$json.email` in Node C would reflect Node B's output, not Node A's original data.

Now imagine Node B doesn't carry the email field forward at all — maybe it only outputs `signup_date`. In that case, from Node C, `$json.email` returns `undefined`. To reach back to Node A directly regardless of what Node B did, you'd write:

```
{{ $node["Node A"].json.email }}
```

That expression skips straight past Node B and pulls `email` directly from Node A's output, no matter how many nodes sit between them.

Intuition

The mental model that makes this click: think of `$json` as the word "this." When you say "this" while pointing at something on a table, you're not describing a memory of what used to be there — you're describing what's physically in front of you right now. In n8n, `$json` inside a node always means "the data sitting in this node right now," which is whatever the immediately preceding node just handed off.

That's the key insight experienced automation builders internalize fast: `$json` is not a global variable that floats around the entire workflow holding one fixed value. It's *scoped* to wherever you're asking from. As an item moves from Node A to Node B to Node C, the meaning of `$json` slides along with it, resetting at every stop to mean "whatever landed here."

This explains why the hardcoded email bug happens in the first place — someone treated a field like a sticky note (write once, stays forever) when they needed a window (`{{ }}`) that reopens and re-evaluates on every run.

It also explains why you sometimes need more reach than `$json` gives you. If you're three nodes downstream and need a value from the very first node — not the immediate parent — `$json` can't see that far. It only ever sees what the previous node produced. That's the gap `$node` is built to close: it lets you name any earlier node by its exact name and pull its data directly, bypassing everything in between.

Brute Force Solution

There isn't really a "brute force" algorithm here in the traditional coding-interview sense, but there is a brute-force *habit* worth naming, because it's exactly what causes the bug in this lesson.

Idea: Type the value you need directly into the field, using whatever the current test data happens to be.

Algorithm: 1. Look at the sample data during testing. 2. Copy the value you see (an email, an ID, a name) directly into the field as plain text. 3. Move on.

Advantages: - Works immediately, no syntax to learn. - Fine for one-off, single-use configuration values that genuinely never change (an API key stored in credentials, a fixed Slack channel name).

Disadvantages: - Breaks the moment the input data changes, which for anything user-facing is every single run. - Fails silently — no error, no warning, just wrong behavior. - Impossible to spot in a review unless you already know to check every field for hardcoded values.

Complexity: Not applicable in time/space terms, but in maintenance terms this is O(n) manual fixes, where n is the number of times someone new signs up and gets the wrong email.

Optimal Solution

The optimal approach is to treat every field that should reflect live data as an expression, and to pick the right variable for how far back you need to reach.

Step 1 — Turn the field into an expression. Next to nearly every field in n8n is a small function icon. Clicking it switches the field from a fixed value into expression mode — the border typically changes color to confirm it's now live.

Step 2 — Open the expression with `{{ }}`. Everything inside the double curly braces is evaluated fresh on every execution as a small JavaScript snippet.

Step 3 — Decide how far back you need to reach.

Situation	Use	Why
You need data from the node immediately before this one	`$json.fieldName`	Shorter, and it's really just a convenient shortcut for referencing the immediate parent
You need data from a node two or more steps back	`$node["Node Name"].json.fieldName`	`$json` only ever sees the immediate parent; `$node` names any earlier node directly

Step 4 — Verify with the live preview. Directly beneath the expression input, n8n shows the resolved value using real data from the last execution. This is the single most underused feature in the editor, and it's what catches typos before they become production bugs.

Step 5 — Combine expressions when needed. A field isn't limited to one expression. You can write `{{ $json.first_name }} {{ $json.last_name }}` in the same field and n8n stitches the results into one string — useful for building full names, combined messages, or composite identifiers.

Python Code

n8n expressions run in JavaScript inside the tool itself, but the underlying resolution logic — "look up a field on the current item, or reach back to a named earlier step" — is worth modeling in Python, since it's the same pattern you'll implement anytime you're building a pipeline with scoped context per stage.

```
from dataclasses import dataclass, field


@dataclass
class NodeOutput:
    """Represents the data produced by a single workflow node."""
    name: str
    json_data: dict = field(default_factory=dict)


class WorkflowContext:
    """
    Mimics n8n's expression scoping: $json resolves against the
    immediately preceding node, while $node resolves against any
    named node executed earlier in the chain.
    """

    def __init__(self):
        self._history: list[NodeOutput] = []
```

```
    def run_node(self, name: str, json_data: dict) -> NodeOutput:
        output = NodeOutput(name=name, json_data=json_data)
        self._history.append(output)
        return output

    def current_json(self, field_name: str):
        """Equivalent to $json.fieldName — looks at the last executed node."""
        if not self._history:
            raise LookupError("No node has run yet")
        return self._history[-1].json_data.get(field_name)

    def node(self, name: str, field_name: str):
        """Equivalent to $node["Name"].json.fieldName — reaches back by name."""
        for output in self._history:
            if output.name == name:
                return output.json_data.get(field_name)
        raise LookupError(f"No node named {name!r} has run")


def demo():
    ctx = WorkflowContext()
    ctx.run_node("Node A", {"email": "maya@example.com", "first_name": "Maya"})
    ctx.run_node("Node B", {"signup_date": "2026-08-10"})  # email dropped here
    ctx.run_node("Node C", {})

    # $json in Node C sees Node B's output only — no email field
    assert ctx.current_json("email") is None

    # $node reaches back to Node A directly, skipping Node B
    assert ctx.node("Node A", "email") == "maya@example.com"

    print("All assertions passed.")


if __name__ == "__main__":
    demo()
```

Code Walkthrough

- `NodeOutput` is a small record type representing what one node produced — its name and its resulting data. This mirrors n8n's internal model where every node execution carries its own JSON payload.
- `WorkflowContext._history` is an ordered list of every node that has run so far, in execution order. This is what makes "reach back to an earlier node" possible at all — nothing is discarded.
- `run_node` simulates a node executing and appending its output to history, the same way n8n passes an item from one node to the next.

- `current_json` implements `$json` semantics: it always looks at `self._history[-1]`, the most recently executed node. It has no concept of anything further back — exactly like the real `$json` variable.
- `node` implements `$node["Name"]` semantics: it searches the full history by name, so it can retrieve data from any point in the chain regardless of how many nodes ran afterward.
- The `demo()` function is a self-check: it proves that `current_json` correctly returns `None` once an upstream field is dropped, and that `node` still finds it by going directly to the source.

Dry Run

Walking through the `demo()` function step by step:

1. `ctx.run_node("Node A", {...})` — history is now `[Node A]`. Node A's data includes `email` and `first_name`.
2. `ctx.run_node("Node B", {"signup_date": "..."})` — history is now `[Node A, Node B]`. Node B's output does **not** include `email`; it only carries `signup_date` forward.
3. `ctx.run_node("Node C", {})` — history is now `[Node A, Node B, Node C]`. Node C produces no fields of its own yet.
4. `ctx.current_json("email")` — this checks `self._history[-1]`, which is Node C's empty dict. `email` isn't there, so it returns `None`. This mirrors what you'd see in the live preview as `undefined` when `$json.email` is used in a field on Node C, given that Node B never forwarded it.
5. `ctx.node("Node A", "email")` — this ignores position entirely and searches by name, finding Node A directly. It returns `maya@example.com`.
6. Both assertions pass, confirming the scoping behavior matches n8n's actual `$json` vs `$node` distinction.

Complexity Analysis

- **`current_json`**: O(1) time — it only ever inspects the last element of the history list. Space is O(1) beyond the existing history.
- **`node`**: O(n) time in the worst case, where n is the number of nodes executed so far, since it may need to scan the full history to find a match by name. Space is O(1) beyond the existing history.
- **Overall workflow context**: O(n) space, where n is the number of nodes executed, since every node's output is retained for the lifetime of the run. This matches n8n's actual behavior — it keeps prior node outputs available precisely so `$node` lookups work no matter how far back they reach.

These are correct because the underlying requirement is exactly this: `$json` needs only the most recent state (constant-time lookup), while `$node` needs the ability to address any prior state by identity (linear-time lookup by name, bounded by however many nodes have run).

Alternative Solutions

A reasonable alternative in n8n itself — rather than in the modeling code above — is using a **Set** or **Edit Fields** node earlier in the chain to explicitly carry forward any fields you'll need later, rather than relying on `$node` to reach back through the whole chain. This trades a slightly longer workflow for expressions that only ever need `$json`, which some teams prefer because it keeps every field lookup local and easy to reason about without knowing the full upstream history. The tradeoff: more nodes to maintain, versus fewer but more far-reaching expressions.

Edge Cases

- **Empty input**: If the triggering node produces no data at all (e.g., a webhook with an empty body), `$json.anything` resolves to `undefined` rather than throwing — this is the exact same silent-failure behavior as the typo case, so treat missing-field checks with the same care.
- **Duplicate node names**: n8n requires unique node names, but if you rename a node after writing a `$node["OldName"]` expression, that expression breaks silently on next run — always re-check expressions after renaming nodes.
- **Minimum case — single node workflow**: `$json` and `$node["OnlyNode"]` resolve identically since there's nothing else to distinguish them from.
- **Maximum case — long chains**: In workflows with dozens of nodes, prefer `$node` sparingly and consider explicitly forwarding critical fields, since deeply nested `$node` references become hard to audit visually.
- **Fields with the same name at different nodes**: If both Node A and Node B produce a field called `status` with different values, `$json.status` in Node C always reflects Node B's version, not Node A's — a frequent source of confusion when refactoring workflows.

Common Mistakes

1. **Leaving a field as a fixed value when it needs to track live data** — the exact bug this lesson opens with.
2. **Typos in field names inside expressions** (`emial` instead of `email`) — n8n returns `undefined` with no error, so the mistake surfaces downstream in a way that looks unrelated to the actual cause.

3. **Assuming `$json` can reach arbitrarily far back** — it only ever reflects the immediately preceding node's output.
4. **Skipping the live preview panel** — trusting memory of a field name instead of glancing at the resolved value before running the whole workflow.
5. **Forgetting to switch a field into expression mode first** — typing `{{ $json.email }}` into a plain fixed field just stores it as literal text instead of evaluating it.
6. **Referencing a node by name after renaming it** — `$node["Old Name"]` silently fails to resolve once the node has been renamed.
7. **Not checking whether an intermediate node dropped a field** — assuming data "just flows through" when a transform node in between may have filtered it out.

Interview Questions

- How would you explain variable scoping in a multi-step pipeline to someone with no programming background?
- What's the tradeoff between referencing data by relative position (like `$json`) versus by explicit identity (like `$node["Name"]`)?
- How would you design a system to warn users when an expression references a field that doesn't exist in the current data, instead of failing silently?
- If you were building the expression preview feature from scratch, what data would you need available at edit time to make it accurate?
- How does this scoping problem compare to variable shadowing or closures in general-purpose programming languages?

Similar Problems

- **LeetCode 146 — LRU Cache**: Similar in spirit because it requires tracking "what's most recent" (like `$json`) versus needing to look up arbitrary earlier entries by key (like `$node`), with different complexity tradeoffs for each access pattern.
- **LeetCode 232 — Implement Queue using Stacks**: Reinforces the idea of state changing shape as it moves through sequential processing stages, similar to how an item's `$json` shape changes as it moves node to node.
- **LeetCode 705 — Design HashMap**: Directly relevant to how `$node`'s name-based lookup would actually be implemented efficiently at scale, using a hash-based structure instead of a linear scan.
- **System design: event pipelines / pub-sub systems**: The same "what does 'current context' mean at each processing stage" question shows up in any streaming or event-driven architecture,

not just no-code tools.

Key Takeaways

- A fixed field freezes a value once; an expression recalculates it on every run.
- `$json` always means "the data at this exact node, right now" — it's scoped to your current position, not global to the workflow.
- To reach further back than the immediate parent node, use `$node["Name"]` to pull data directly from any earlier step.
- n8n fails silently on missing fields — always trust the live preview over memory when writing expressions.
- You can combine multiple expressions in a single field to build composite values like full names or messages.

Watch the Video

This lesson is easier to absorb watching it happen live — seeing the expression editor, the live preview, and the exact moment `$json` resets between nodes. Watch the full walkthrough here: {LINK}

About the Series

This article is part of **Fun with Learning Technology**, a series built around teaching real automation, backend, and software engineering concepts the way a senior engineer would explain them to a teammate — grounded in the intuition behind why something works, not just the steps to make it work. Whether the topic is n8n, Python, SQL, or classic interview algorithms, the goal is always the same: build a mental model strong enough that you can solve the next unfamiliar problem on your own.

Call To Action

If this cleared up something that's been quietly confusing you in n8n, do three things: like the video on YouTube so more builders see it, subscribe to the channel for the rest of the series, and subscribe to the Substack newsletter for written breakdowns like this one delivered straight to your inbox. Drop a comment with the workflow bug that cost you the most debugging time — there's a good chance it'll become the next lesson. And if this saved you from shipping a hardcoded field, share it with the teammate who's about to make the same mistake.

Quick Review

n8n: {{ }} in a field means what?

Expression mode — field value is evaluated as JavaScript, not read as literal text.

What does $json refer to inside an expression?

The JSON data of the current item on the current node's input — changes meaning per node.

Why does $json 'keep changing meaning' across nodes?

$json always resolves relative to the node you're in, so the same expression pulls different data depending on placement.

How do you pull data from a node further back than the previous one?

Use $node["Node Name"].json instead of $json, referencing the node explicitly by name.

Common 'ticking time bomb' bug with expressions?

Field left in plain-text mode while typing {{ }} — syntax never evaluates, silently passes literal string downstream.

Static text field vs expression field — key difference?

Static: fixed value every run. Expression: recalculated per execution from live upstream item data.